

PCP – CA2 – Syllabus & Pattern

React Frontend Assessment

State Management, Routing, and Data Handling

1. Objective

This assessment evaluates your understanding of:

- React fundamentals (components, props, state)
- React Router (navigation, params)
- Context API (global state management)
- useReducer (complex state handling)
- API consumption using Axios / fetch
- Component-based architecture
- Data rendering and transformation (map, filter, reduce)
- Form handling and validation
- Clean UI structuring

2. Assessment Overview

You will be provided with:

- A **Public API** → to get a token
- A **Private API** → to fetch dataset

Your task is to:

1. Fetch a token using your student credentials
2. Use the token to fetch dataset
3. Store dataset in **global state (Context + Reducer)**
4. Build a React application with required features
5. Implement routing using React Router
6. Deploy your application (Vercel / Netlify)
7. Submit GitHub + Live URL

3. Step 1 – Obtain Token (Public API)

Endpoint

POST /public/token

Request Body

```
{
  "studentId": "YOUR_REGISTER_NUMBER",
  "password": "YOUR_PASSWORD"
}
```

Password will be shared via email.

Response

```
{
  "token": "JWT_TOKEN"
}
```

Rules

- Must use POST request
- Must use Axios / fetch
- Token validity is limited

4. Step 2 – Fetch Dataset (Private API)

Endpoint

```
GET /private/data
```

Header

```
Authorization: Bearer <JWT_TOKEN>
```

Behavior

- Valid token → dataset returned
- Invalid/expired token → 401

5. Implementation Rules (STRICT)

Mandatory

1. Fetch data **only once (on app load)**
2. Store data using **Context + Reducer**
3. Use global state across all components
4. Use React Router for navigation

Not Allowed

- Fetching API inside multiple components
- Using only local state instead of global state
- Hardcoding dataset
- Skipping Context or Reducer
- Sharing credentials

6. Required Application Structure

```
src/  
├── components/  
├── pages/  
├── context/  
│   └── AppContext.jsx  
├── reducer/  
│   └── AppReducer.js  
├── router/  
│   └── AppRouter.jsx  
├── services/  
│   └── api.js  
└── App.jsx
```

7. Features to Implement

(Based on Movie Wishlist App Model)

Q1 – Add Movie (Form with Validation)

Requirements:

- Form using useState
- Validate:
 - Movie name → letters only
 - Year → 4 digits
- Add movie to global state (Context + Reducer)
- Prevent invalid submission

Q2 – Movie List (CRUD Operations)

Requirements:

- Display all movies using .map()
- Add new movie (Create)
- Delete movie (Delete)
- Toggle watched status
- Use reusable MovieCard component

Q3 – Favorites (Context + Routing)

Route:

/favorites

Requirements:

- Add/remove favorites
- Store in global state
- Display favorite movies



Q4 – Navigation (React Router)

Routes:

/ → Home

/favorites → Favorites

/stats → Stats

Requirements:

- Proper routing setup
- Navigation between pages
- Active route handling

Q5 – Stats Dashboard (Analysis)

Route:

/stats

Requirements:

Use `.filter()` and `.reduce()` to compute:

- Total movies
- Watched movies
- Movies grouped by genre

Must be dynamically calculated

8. Technical Requirements

Your project must include:

- Functional Components
- `useState`
- `useEffect`
- `useContext`
- `useReducer`
- React Router
- Axios / `fetch`
- Form validation
- Clean folder structure

9. Deployment Requirements

You must:

1. Deploy using:
 - Vercel / Netlify
2. Ensure:
 - App runs without errors

- Routing works correctly
- No console errors

3. Submit:

- GitHub repository
- Live deployed URL

10. Evaluation Criteria (100 Marks)

Criteria	Marks
API Integration (token + data)	15
Context + Reducer Implementation	20
Routing (React Router)	15
Feature Implementation (Movie App)	25
State Management Logic	10
Code Structure & Cleanliness	10
Deployment	5

11. Academic Integrity

- Each student gets unique credentials
- Token expires after limited time
- Sharing credentials → malpractice
- Hardcoding data → disqualification

12. Key Learning Mapping

Concept	Usage
useEffect	API fetch
Context	Global state
Reducer	State updates
Router	Navigation
map/filter/ reduce	Logic
Props	Component reuse